Lab 3 DNS and Kaminsky Attack

Network Security (50.020) | Timothy Seah | Student ID 1007654 | October 2026

I used four lab containers to trace normal DNS resolution, construct a DNS query and forged response, test repeated-response attacks, and monitor the attack traffic.

1 Network and basic operations

Node	Address	Role
node1	10.11.2.5	Client generating DNS queries
node2	10.11.2.6	Recursive and caching DNS resolver
node3	10.11.2.7	Analyst and attack-script host
node4	10.11.2.8	Lab authoritative DNS and web server

Docker Compose showed all four containers running. Before the lookups, node2's cache contained root nameserver information but no relevant example.com answer.

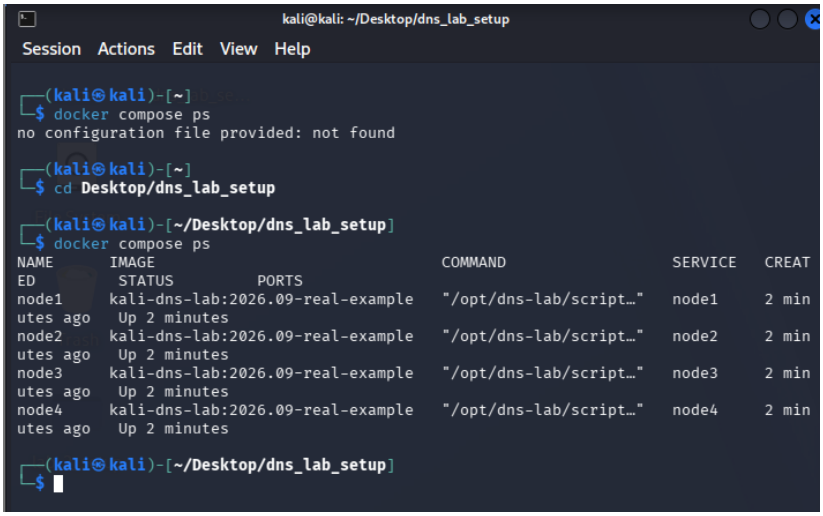


Figure 1. Four lab containers running under Docker Compose.

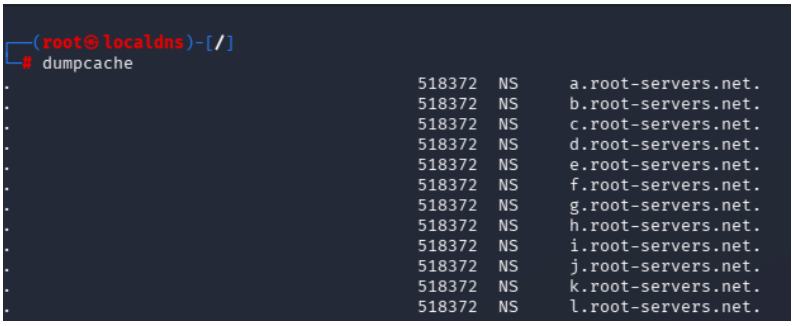


Figure 2. Initial cache includes root nameserver records but no example.com answer.

2 Normal DNS resolution and cache

Command on node1	Server contacted	Purpose and observed result
dig example.com	node2	Recursive public lookup example.com A records resolved to 104.20.23.154 and 172.66.147.243.
dig ns.attackerpage.com	node2	Lookup of lab nameserver response gave A 10.11.2.8 and an attackerpage.com NS record
dig @ns.attackerpage.com example.com	node4 directly	Explicit query to lab authority Returned example.com A 10.11.2.8 without recursion

These answers came from different DNS paths. Node2 resolved the public example.com domain through the Internet, while node4 answered from a separate lab-controlled zone. The direct query to node4 therefore does not show that node2's cache was poisoned.

Resolver lookup through node2

```
┌──(root㉿user)-[/]
└─# dig ns.attackerpage.com

; <<>> DiG 9.20.29-1-Debian <<>> ns.attackerpage.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 44023
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 35b990440ad9a6b9010000006ac7375c228cc52c293836dc (good)
;; QUESTION SECTION:
;ns.attackerpage.com.           IN      A

;; ANSWER SECTION:
ns.attackerpage.com.    300     IN      A       10.11.2.8

;; AUTHORITY SECTION:
attackerpage.com.       300     IN      NS      ns.attackerpage.com.

;; Query time: 8 msec
;; SERVER: 10.11.2.6#53(10.11.2.6) (UDP)
;; WHEN: Thu Oct 08 06:25:32 UTC 2026
;; MSG SIZE  rcvd: 106
```

Direct query to node4

```
┌──(root㉿user)-[/]
└─# dig @ns.attackerpage.com example.com

; <<>> DiG 9.20.29-1-Debian <<>> @ns.attackerpage.com example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17729
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b97164587eb78f0b010000006ac73767b98b3ed123fe0eff (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            300     IN      A       10.11.2.8

;; AUTHORITY SECTION:
example.com.            300     IN      NS      ns.attackerpage.com.

;; Query time: 0 msec
;; SERVER: 10.11.2.8#53(ns.attackerpage.com) (UDP)
;; WHEN: Thu Oct 08 06:25:43 UTC 2026
;; MSG SIZE  rcvd: 114
```

Figure 3. Recursive lookup through node2 and direct lookup to node4.

After the queries, node2's cache contained public A and NS records for example.com, as well as the A record for ns.attackerpage.com and the NS delegation for attackerpage.com. These records remain until their TTLs expire, so node2 can answer a repeat query without another upstream lookup.

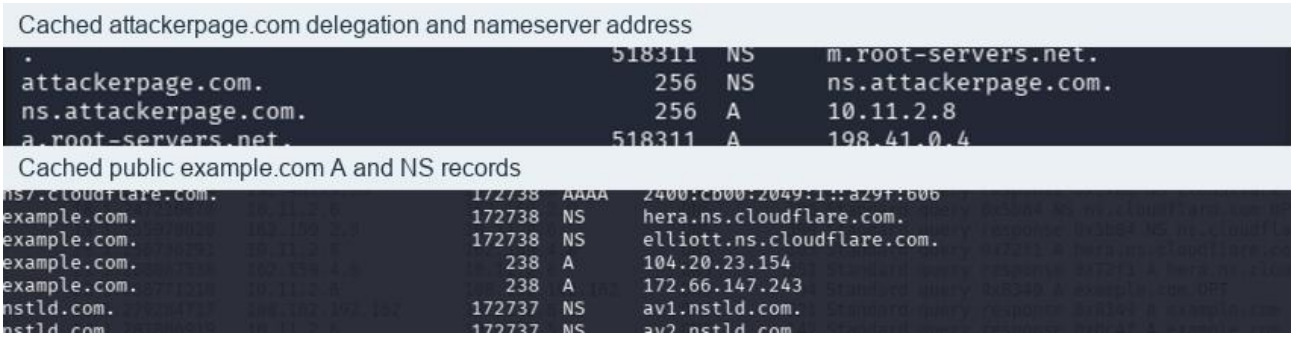


Figure 4. Selected records in node2's cache after the lookups.

The four captures show node1 querying node2, node2 contacting public DNS for example.com, and node4 receiving the query sent directly to it. Node3 was not involved in these lookups.

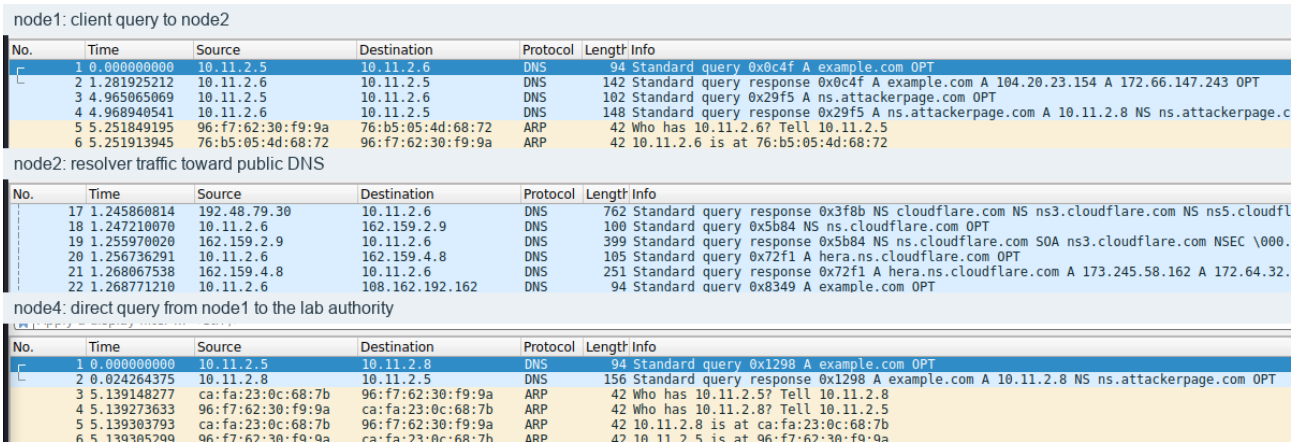


Figure 5. DNS traffic seen at the client, resolver, and lab authority.

Wireshark shows two separate DNS transactions for example.com. Node1 queried node2 with TXID 0x0c4f; node2 then queried upstream with TXID 0x8349. After receiving the matching upstream reply, node2 returned the answer to node1 using the original TXID, 0x0c4f. Each request has its own ID, so the two IDs do not need to match.

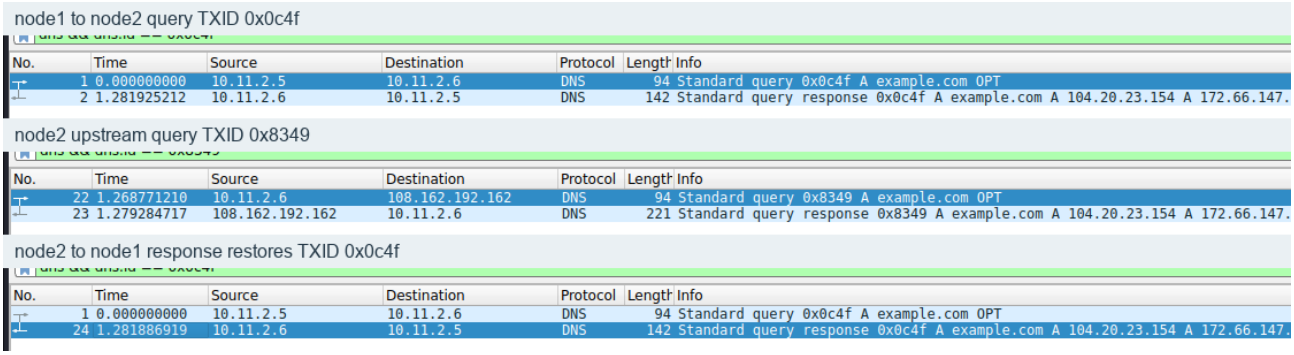


Figure 6. Client and resolver transaction IDs, followed by the client-facing reply.

3 Construct DNS query and response

From node3, I used dns_query.py to ask node2 for the A record of ns.attackerpage.com. The query went from 10.11.2.7:40396 to 10.11.2.6:53, and the reply returned to the same client port. Both packets used TXID 0x2c84. The successful reply contained the A record 10.11.2.8 and an NS record for attackerpage.com, each with a TTL of 300 seconds.

Python query and matching TXID

```
┌──(root㉿analyst)-[~]
└─# python3 /root/dns_query.py ns.attackerpage.com
Query: 10.11.2.7:40396 → 10.11.2.6:53
Transaction ID: 0×2c84
###[ DNS ]###
  id        = 11396
  qr        = 0
  opcode    = QUERY
  aa        = 0
  tc        = 0
```

Response A and NS records

```
   |  unicastresponse= 0
   |  qclass    = IN
  \an        \
   |###[ DNS Resource Record ]###
   |  rrname    = b'ns.attackerpage.com.'
   |  type      = A
   |  cacheflush= 0
   |  rclass    = IN
   |  ttl       = 300
   |  rdlen     = None
   |  rdata     = 10.11.2.8
  \ns        \
   |###[ DNS Resource Record ]###
   |  rrname    = b'attackerpage.com.'
   |  type      = NS
   |  cacheflush= 0
   |  rclass    = IN
   |  ttl       = 300
   |  rdlen     = None
   |  rdata     = b'ns.attackerpage.com.'
  \ar        \

Received a matching DNS response with answer records.

┌──(root㉿analyst)-[~]
```

Figure 7. Python query output and matching DNS answer on node3.

node3 observes the DNS query

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.2.7	10.11.2.6	DNS	79	Standard query 0x2c84 A ns.attackerpage.com
3	0.096182606	10.11.2.6	10.11.2.7	DNS	109	Standard query response 0x2c84 A ns.attackerpage.com A 10.11.2.8

node2 observes the DNS query

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.2.7	10.11.2.6	DNS	79	Standard query 0x2c84 A ns.attackerpage.com
6	0.096147926	10.11.2.6	10.11.2.7	DNS	109	Standard query response 0x2c84 A ns.attackerpage.com A 10.11.2.8

node3 observes the DNS response

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.2.7	10.11.2.6	DNS	79	Standard query 0x2c84 A ns.attackerpage.com
3	0.096182606	10.11.2.6	10.11.2.7	DNS	109	Standard query response 0x2c84 A ns.attackerpage.com A 10.11.2.8

node2 observes the DNS response

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.2.7	10.11.2.6	DNS	79	Standard query 0x2c84 A ns.attackerpage.com
6	0.096147926	10.11.2.6	10.11.2.7	DNS	109	Standard query response 0x2c84 A ns.attackerpage.com A 10.11.2.8

Figure 8. The same query and response observed on node3 and node2.

For a forged reply to work, it must match an outstanding query's name, transaction ID, addresses, and UDP ports, and arrive before the legitimate reply. The forged reply for gwgsz.example.com claimed A 66.66.66.66 and NS ns.attackerpage.com. Node2 saw it at 88.293861702 s, before sending its upstream query at 91.634874463 s. Its TXID was 0x0000, while the upstream query used 0x3ff4. The legitimate reply arrived at 91.651906537 s, and node2 returned a negative SOA response. The cache was not poisoned.

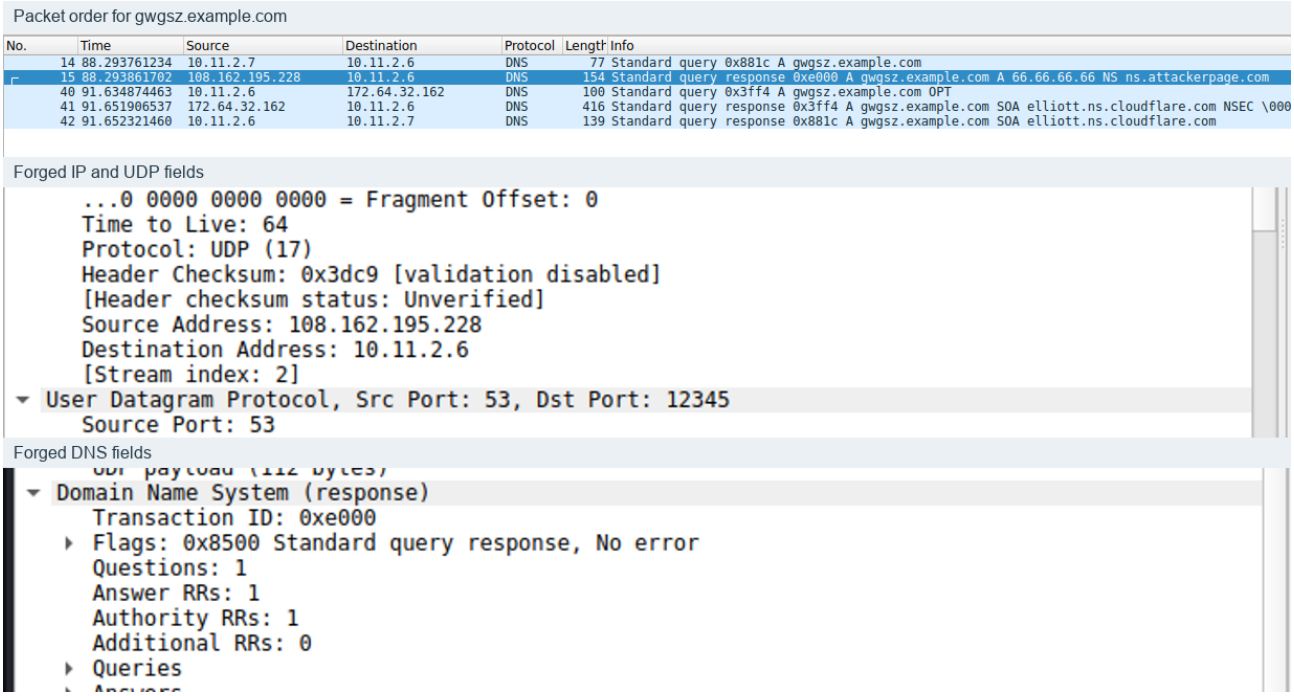


Figure 9. Forged reply timing and packet fields compared with the legitimate exchange.

4 Kaminsky attack demonstration

I ran the instructor-provided attack.py on node3 with 20, 50, and 200 forged replies in separate captures, using a fresh random subdomain each time. All forged replies in each run reached node2 before the legitimate upstream reply.

Replies	Fresh name	Upstream destination	Query to legitimate reply	Forged replies first
20	vnztg.example.com	173.245.58.162	0.000312 to 0.090795 s	20
50	dfpbg.example.com	108.162.192.162	0.000348 to 0.043553 s	50
200	xjypw.example.com	172.64.32.162	0.000466 to 0.047623 s	200

Node2 sent its upstream queries to 173.245.58.162, 108.162.192.162, and 172.64.32.162. The forged replies claimed to come from 108.162.195.228, which matched none of those destinations. In all three runs, node2 returned the legitimate negative SOA answer. I found no evidence of cache poisoning.

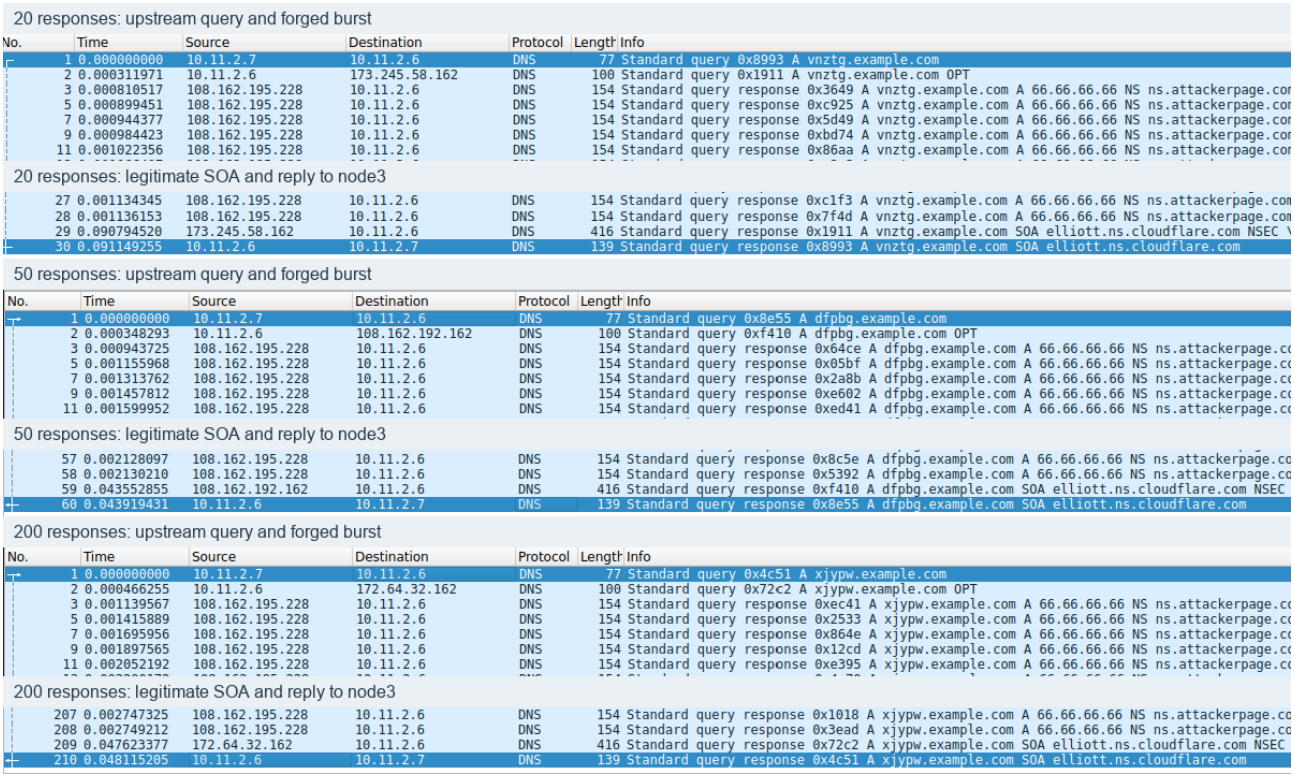


Figure 10. Upstream query, forged-response burst, and legitimate SOA reply for each run.

If each forged reply guesses a random 16-bit TXID, the chance of at least one match is (1-(1-1/65536) ^ N): ~0.0305% with 20 replies, ~0.0763% with 50, and ~0.3047% with 200. More replies improve the chance of an ID match, but a valid reply must also match the upstream server address, UDP ports, and question, and arrive at the right time. In these runs, the forged source address did not match the upstream server, and node2 accepted none of the replies. I did not modify the lab code.

5 AI assisted DNS attack detection

With AI assistance, I developed dns_attack_detector.py to monitor upstream DNS queries and replies to node2's UDP port 12345. Over a two-second window, it counts replies and distinct TXIDs for each name. It alerts when at least five replies use different TXIDs without matching an observed query. When it sees the upstream query, it also checks the reply's source IP and TXID.

During the 50-reply run for ijjgi.example.com, the detector alerted after seeing five distinct transaction IDs. By the end, it had counted 50 replies, each with a different ID, and none matched a query it had observed. Wireshark showed that the replies advertised 66.66.66.66 and ns.attackerpage.com. The detector had not seen the expected upstream server, so this alert shows that it detected the burst of replies. It does not confirm a source IP mismatch or prove that node2's cache was poisoned. The detector only reports suspicious traffic and does not block it.

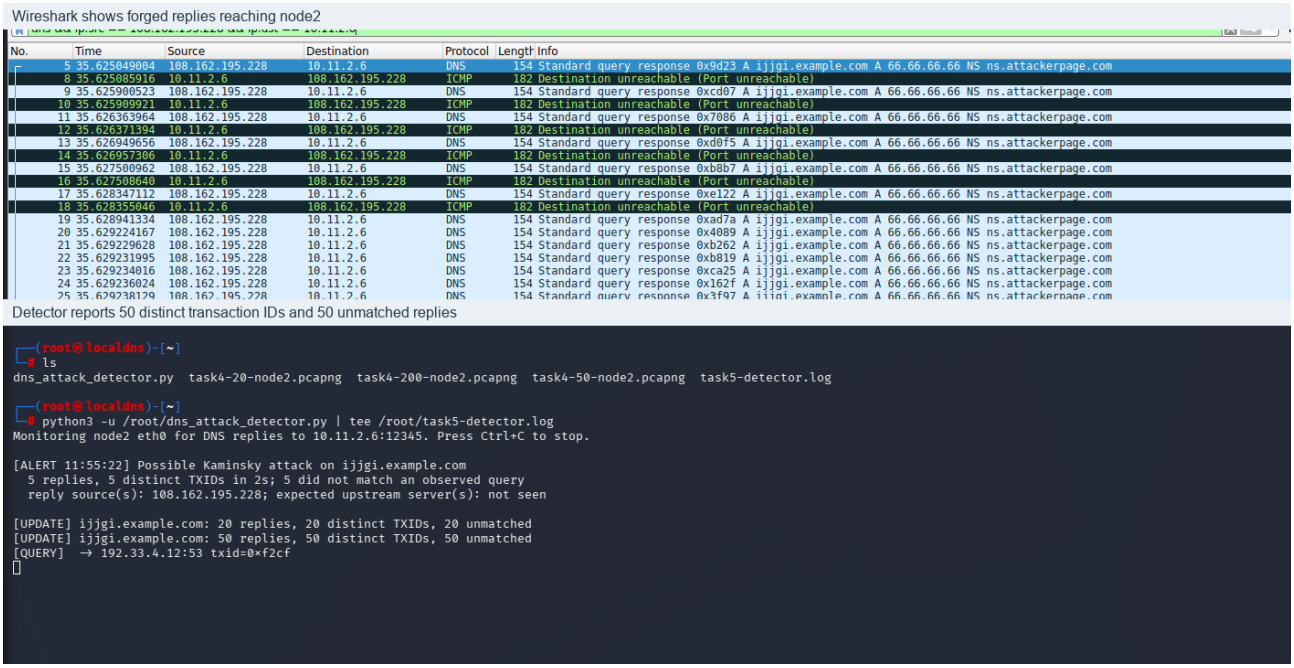


Figure 11. Forged DNS burst beside the detector alert on node2.

AI helped draft the query program and detector, and I used Wireshark to verify their behavior. The captures showed that arriving early is not enough for a forged reply to succeed, and that it must match an outstanding query's IP, UDP, and DNS details too. The detector would also need testing with normal traffic to measure false positives.